UNIVERSITY OF PASSAU

6061 | Introduction to Deep Learning

Exercise 1

Prof. Dr. Florian Lemmerich[1] Andreas Einwiller[1] Max Klabunde[1*]

[1]Professorship for Applied Machine Learning
University of Passau

[*](2021–2025) Preparation, creation, and presentation

15.10.2026

About Me

- Andreas Einwiller
- PhD student / Research Assistant
- AuAu[1,2]: Auditing Authoritarian Alignment
- M.Sc. Computer Science & AIE from the University of Passau
- Teaching this course since 1991[3]
- Contact: andreas.einwiller@uni-passau.de



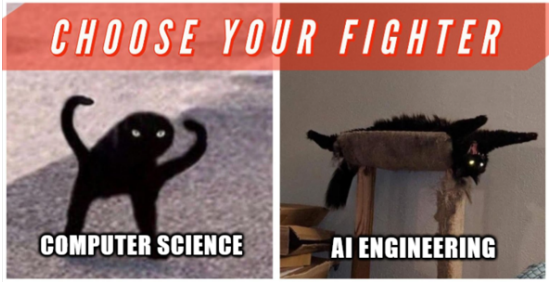
[1] https://arxiv.org/abs/2606.16127

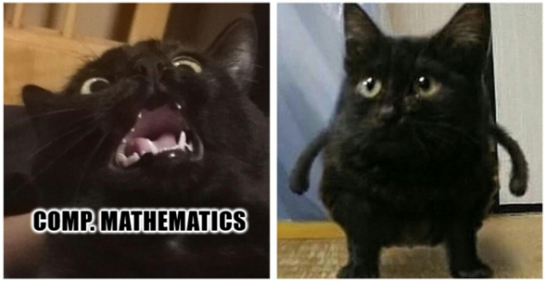
[2] https://en.bidt.digital/research-project/authoritarian-ai-how-large-language-models-llms-align-with-russias-propaganda/

[3] https://people.idsia.ch/~juergen/deep-learning-history.html (sorry Juergen)

Know Your Audience

What's your primary field of study?





Organization

- Room: SR 029 (WIWI)
- Time: We offer two sessions each Thursday
  - 12:15–13:45
  - 14:15–15:45
- Both have the same content, help us to balance them
- Join *one* group in Stud.IP: participants $\rightarrow$ groups
- (Almost) all course materials will be available on Stud.IP

Provisional Schedule

Week	Content	Note	Homework
1	Introduction to Python		Coding
2	Introduction to Pytorch		Coding
3	Gradient Descent, Evaluation Metrics		Pen and Paper (PnP)
4	Automatic Differentation, Stochastic Gradient Descent		Coding, PnP
5	Multi-Layer Perceptrons		Coding
6	Loss Functions, Advanced Programming	Test 1	PnP
7	Activation Functions, Parameter Initialization		Coding, PnP
8	Improving Training with Optimizers and Regularization		Coding
9	Convolutional Neural Networks I		PnP
10	Convolutional Neural Networks II		
	Christmas Break		
11	Recurrent Neural Networks	Test 2	Coding, PnP
12	Word Embedding		Coding, PnP
13	Transformer		PnP
14	Pretrained Models, Autoencoders, Generative Adversarial Networks		Coding, PnP
15	Graph Neural Networks, Interpretability, (Recap)		

Tests

- Do *not* affect your final grade
- Shown to improve learning [Roediger et al., 2011]
- Additional preparation for the exam

A Typical Session

UNIVERSITY OF PASSAU

- During class:
  - If needed: discussion of last week's homework
  - Detailed worked examples
- After class:
  - Take the time needed and solve the homework! (1 ECTS :: 30h)
  - Mix of Pen and paper and coding exercises
  - Recommendation: Do the homework the day after the exercise
  - Homework is not graded

Ask questions!

- In class: anytime
- Afterwards: Stud.IP forum (non-anonymous) $\gg$ E-Mail
- Let others benefit from your questions!

Feedback is always welcome

- Don't wait to give feedback
- The official course evaluation will be too late (to help u)
- Give anonymous feedback for today at
  https://lgms.nl/p/presentation-feedback/?code=263dd9
  (QR Code at the end)

Section 1

Introduction to Python

Preface

Assumptions:

- You know how to program ($\leftarrow$ fundamentals ✓)
- The main barrier to using Python is unfamiliar syntax

Goal of this introduction:

- Familiarity with syntax
- Overview of builtins
- Overview of ecosystem
- You can write simple functions, control flow, and know where to look up documentation for packages.

Doing the homework is key!

Why Python?

UNIVERSITY OF PASSAU

- Simple (?)
  - High-level, almost looks like “pseudo code”
  - Interpreted: no compile step, runs on all major platforms
  - Dynamically typed
  - Automatic memory management (garbage collection)
- Powerful
  - Large standard library
  - Huge ecosystem
- Extensible: Integrate C code for critical parts
- **Almost everything related to ML is using Python**
- Never intended for AI, can we do better? Maybe: Tensor Logic

Why Python?

UNIVERSITY OF PASSAU

- Simple (?)
  - High-level, almost looks like “pseudo code”
  - Interpreted: no compile step, runs on all major platforms
  - Dynamically typed
  - Automatic memory management (garbage collection)
- Powerful
  - Large standard library
  - Huge ecosystem
- Extensible: Integrate C code for critical parts
- **Almost everything related to ML is using Python**
- Never intended for AI, can we do better? Maybe: Tensor Logic

Why Python?

UNIVERSITY OF PASSAU

- Simple (?)
  - High-level, almost looks like “pseudo code”
  - Interpreted: no compile step, runs on all major platforms
  - Dynamically typed
  - Automatic memory management (garbage collection)
- Powerful
  - Large standard library
  - Huge ecosystem
- Extensible: Integrate C code for critical parts
- **Almost everything related to ML is using Python**
- Never intended for AI, can we do better? Maybe: Tensor Logic

Simple Programs

UNIVERSITY OF PASSAU

program1.py

```
1 a = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
2 print(sum(a))
```

- Indentation matters
- Here 2 spaces, but typically 4 spaces. Needs to be consistent within a code block.

Simple Programs

UNIVERSITY OF PASSAU

program1.py

```
1 a = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
2 print(sum(a))
```

program2.py

```
1 for i in range(10):
2   print(i)
```

- Indentation matters
- Here 2 spaces, but typically 4 spaces. Needs to be consistent within a code block.

Simple Programs

UNIVERSITY OF PASSAU

program1.py

```
1 a = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
2 print(sum(a))
```

program2.py

```
1 for i in range(10):
2   print(i)
```

Invalid program2.py

```
1 for i in range(10):
2 print(i)
```

- Indentation matters
- Here 2 spaces, but typically 4 spaces. Needs to be consistent within a code block.

Simple Programs

UNIVERSITY OF PASSAU

program1.py

```
1 a = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
2 print(sum(a))
```

program2.py

```
1 for i in range(10):
2   print(i)
```

Invalid program2.py

```
1 for i in range(10):
2 print(i)
```

- Indentation matters
- Here 2 spaces, but typically 4 spaces. Needs to be consistent within a code block.

FizzBuzz

- Print numbers from 1 to 100
- For multiples of 3, print “Fizz”
- For multiples of 5, print “Buzz”
- For multiples of **both** 3 and 5, print “FizzBuzz”!

fizzbuzz.py

```python
def fizz_buzz(n: int) -> None:
    for i in range(1, n + 1):
        if i % 3 == 0 and i % 5 == 0:
            print("FizzBuzz")
        elif i % 3 == 0:
            print("Fizz")
        elif i % 5 == 0:
            print("Buzz")
        else:
            print(i)

fizz_buzz(n=100)
```

FizzBuzz

- Print numbers from 1 to 100
- For multiples of 3, print “Fizz”
- For multiples of 5, print “Buzz”
- For multiples of **both** 3 and 5, print “FizzBuzz”!

fizzbuzz.py

```python
def fizz_buzz(n: int) -> None:
  for i in range(1, n + 1):
    if i % 3 == 0 and i % 5 == 0:
      print("FizzBuzz")
    elif i % 3 == 0:
      print("Fizz")
    elif i % 5 == 0:
      print("Buzz")
    else:
      print(i)

fizz_buzz(n=100)
```

FizzBuzz

- Print numbers from 1 to 100
- For multiples of 3, print “Fizz”
- For multiples of 5, print “Buzz”
- For multiples of **both** 3 and 5, print “FizzBuzz”!

fizzbuzz.py

```python
def fizz_buzz(n: int) -> None:
  for i in range(1, n + 1):
    if i % 3 == 0 and i % 5 == 0:
      print("FizzBuzz")
    elif i % 3 == 0:
      print("Fizz")
    elif i % 5 == 0:
      print("Buzz")
    else:
      print(i)

fizz_buzz(n=100)
```

FizzBuzz

- Print numbers from 1 to 100
- For multiples of 3, print “Fizz”
- For multiples of 5, print “Buzz”
- For multiples of **both** 3 and 5, print “FizzBuzz”!

fizzbuzz.py

```python
def fizz_buzz(n: int) -> None:
  for i in range(1, n + 1):
    if i % 3 == 0 and i % 5 == 0:
      print("FizzBuzz")
    elif i % 3 == 0:
      print("Fizz")
    elif i % 5 == 0:
      print("Buzz")
    else:
      print(i)

fizz_buzz(n=100)
```

FizzBuzz

- Print numbers from 1 to 100
- For multiples of 3, print “Fizz”
- For multiples of 5, print “Buzz”
- For multiples of **both** 3 and 5, print “FizzBuzz”!

fizzbuzz.py

```python
def fizz_buzz(n: int) -> None:
  for i in range(1, n + 1):
    if i % 3 == 0 and i % 5 == 0:
      print("FizzBuzz")
    elif i % 3 == 0:
      print("Fizz")
    elif i % 5 == 0:
      print("Buzz")
    else:
      print(i)

fizz_buzz(n=100)
```

FizzBuzz

- Print numbers from 1 to 100
- For multiples of 3, print “Fizz”
- For multiples of 5, print “Buzz”
- For multiples of **both** 3 and 5, print “FizzBuzz”!

fizzbuzz.py

```python
def fizz_buzz(n: int) -> None:
  for i in range(1, n + 1):
    if i % 3 == 0 and i % 5 == 0:
      print("FizzBuzz")
    elif i % 3 == 0:
      print("Fizz")
    elif i % 5 == 0:
      print("Buzz")
    else:
      print(i)

fizz_buzz(n=100)
```

FizzBuzz

- Print numbers from 1 to 100
- For multiples of 3, print “Fizz”
- For multiples of 5, print “Buzz”
- For multiples of **both** 3 and 5, print “FizzBuzz”!

fizzbuzz.py

```python
def fizz_buzz(n: int) -> None:
  for i in range(1, n + 1):
    if i % 3 == 0 and i % 5 == 0:
      print("FizzBuzz")
    elif i % 3 == 0:
      print("Fizz")
    elif i % 5 == 0:
      print("Buzz")
    else:
      print(i)

fizz_buzz(n=100)
```

How to run this code?

```
python fizzbuzz.py
```

- How to get Python on your computer? (See homework!)
- Installation via python.org not recommended
- We will use uv as our environment manager

```
uv run fizzbuzz.py
```

Why an Environment Manager?

UNIVERSITY OF PASSAU

project1.py

```
1 import numpy  # package for matrix manipulation, need version 1.13
```

project2.py

```
1 import numpy  # package for matrix manipulation, need version 2.0
```

- Different dependencies require separated environments
- Might also need different Python versions
- Reproducibility, what's that?

Why an Environment Manager?

project1.py

```
1 import numpy  # package for matrix manipulation, need version 1.13
```

project2.py

```
1 import numpy  # package for matrix manipulation, need version 2.0
```

- Different dependencies require separated environments
- Might also need different Python versions
- Reproducibility, what's that?

Why an Environment Manager?

UNIVERSITY OF PASSAU

project1.py

```
1 import numpy  # package for matrix manipulation, need version 1.13
```

project2.py

```
1 import numpy  # package for matrix manipulation, need version 2.0
```

- Different dependencies require separated environments
- Might also need different Python versions
- Reproducibility, what's that?

Why an Environment Manager?

UNIVERSITY OF PASSAU

project1.py

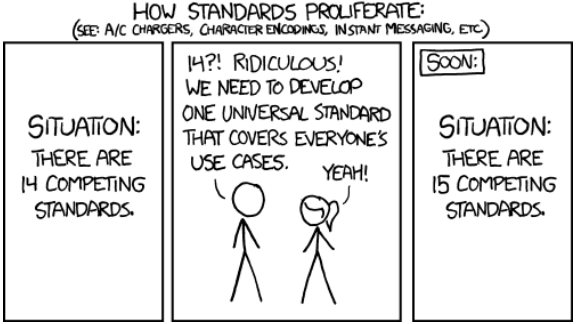
```
1 import numpy  # package for matrix manipulation, need version 1.13
```

project2.py

```
1 import numpy  # package for matrix manipulation, need version 2.0
```

- Different dependencies require separated environments
- Might also need different Python versions
- Reproducibility, what's that?

Many Options for Environment Managers



https://xkcd.com/927/

Jupyter Notebooks: Blessing and Curse

- Run and rerun code while keeping state
- Interleave code with Markdown text and visualizations
- Interactively explore code and data
- Easily leads to spaghetti code and encourages bad coding habits: see https://www.youtube.com/watch?v=7jiPeIFXb6U
- Demo

Jupyter Notebooks: Blessing and Curse

UNIVERSITY OF PASSAU

- Run and rerun code while keeping state
- Interleave code with Markdown text and visualizations
- Interactively explore code and data
- Easily leads to spaghetti code and encourages bad coding habits: see https://www.youtube.com/watch?v=7jiPeIFXb6U
- Demo

Tutorials

- Relatively comprehensive with example problems: https://www.pythonlikeyoumeanit.com/index.html
- Overview, mainly for syntax: https://learnxinyminutes.com/docs/python/
- Search YouTube for cool playlists
- “Pair programming” with your favorite LLM

Next Steps

- Follow the instructions on homework sheet 1 (uploaded to Stud.IP)
- Write simple programs to learn the syntax of Python
- Use common libraries (e.g., *Pandas*, *NumPy*, *Scikit-learn*, *PyTorch*)
- Start now. I'm here to help!

Feedback

Please use the text field



https://lgms.nl/p/presentation-feedback/?code=263dd9